

MULTIMEDIA INTERACTIVE PROJECT

BHUMI RAKSHA

Skies of Tomorrow

SDG GOAL 13: CLIMATE ACTION

*Buku Panduan Teknis, Arsitektur Kode, Mekanik Permainan, dan Logika Program*Disusun Oleh: **PROJECT GREEN ASCENT**

**Abhirama
Balaphradana V. R.**
NIM: 2338018077

M Rizqi Akbar Hidayat
NIM: 2300018409

Afif Hadriansyah
NIM: 2300018400

PROGRAM STUDI S1 INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI · UNIVERSITAS AHMAD DAHLAN
YOGYAKARTA, 2026

Daftar Isi

1. Pendahuluan & Konsep Utama Game	3
2. Struktur Berkas & Scene Navigation	4
3. Analisis Mekanik Core Game	5
3.1 Lokasi Munculnya Musuh (Enemy Spawning)	5
3.2 Mekanik Awan Besar (Special Obstacles)	5
3.3 Pengaturan Spawn Rate	6
3.4 Sistem Pengaturan Kecepatan (Speed Scaling)	6
3.5 Mekanik Tembakan & Amunisi Laser	7
3.6 Mekanik Kemunculan Soal (Question Triggering)	7
3.7 Penalti Jawaban Salah & Tabrakan	8
3.8 Sistem Nyawa & Damage Loop	8
3.9 Konfigurasi Tingkat Kesulitan	9
4. Logika Game (Game Loop) & Detail Kecil	10
4.1 Scrolling Background & Parallax	10
4.2 Hovering Physics & Kontrol Pesawat	10
4.3 Efek Ledakan Partikel Vektor	11
4.4 Sistem Pause Jeda Game	11
5. Kesimpulan & Ringkasan Teknis	12

1. Pendahuluan & Konsep Utama Game

Bhumi Raksha: Skies of Tomorrow adalah sebuah permainan edukatif multimedia interaktif berbasis web 2D. Proyek ini dikembangkan menggunakan pustaka game HTML5 terkemuka, **Phaser.js v3**, yang berjalan secara native di web browser modern tanpa memerlukan instalasi tambahan.

Tema utama yang diangkat dalam game ini adalah **SDG Goal 13: Climate Action (Penanganan Perubahan Iklim)**. Game ini bertujuan memberikan pemahaman praktis dan edukasi interaktif kepada para pemain—khususnya mahasiswa baru pada kegiatan Program Pengenalan Kampus (P2K)—mengenai

pemanasan global, efek rumah kaca, energi terbarukan, polusi udara, serta aksi nyata pelestarian lingkungan hidup.

CORE GAMEPLAY LOOP (ALUR UTAMA PERMAINAN)

Pemain mengendalikan sebuah pesawat ramah lingkungan yang terbang melintasi langit. Terdapat dua jenis rintangan: rintangan biasa (cerobong asap pabrik pencemar) yang menembakkan gas berbahaya, dan rintangan khusus (awan polusi besar). Saat awan besar mendekat, radar pesawat mengaktifkan modul soal edukatif. Menjawab dengan benar akan mengisi ulang amunisi dan menembakkan laser penghancur. Menjawab salah atau tertabrak rintangan akan merusak pelindung pesawat (mengurangi nyawa).

2. Struktur Berkas & Scene Navigation

Proyek game ini dirancang dengan arsitektur bersih menggunakan pemisahan modular berbasis kelas ES6 JavaScript. Struktur berkas utamanya adalah sebagai berikut:

Path Berkas	Komponen Utama	Deskripsi Singkat Fungsi
index.html	Entry Point Web	Tempat inialisasi awal, pemuatan library Phaser.js, dan container CSS wrapper UI.
src/main.js	Inisialisasi Game	Konfigurasi global mesin Phaser (dimensi layar 960x640, mode skala FIT, pendaftaran scene).
src/scenes/MenuScene.js	Scene Menu Utama	Menyediakan HUD futuristik, tutorial permainan, musik latar (BGM), dan tombol pilihan tingkat kesulitan.
src/scenes/GameScene.js	Scene Gameplay Inti	Mengatur loop pembaruan game, pesawat pemain, spawning musuh, proyektil, tabrakan, dan HUD.
src/scenes/QuestionScene.js	Scene Overlay Soal	Popup modal pertanyaan yang menutupi area permainan saat modul soal edukasi aktif.
src/scenes/ScoreScene.js	Scene Skor Akhir	Layar evaluasi hasil permainan, menampilkan skor, persentase akurasi, dan medali gelar pemain.
src/utils/QuestionManager.js	Manajer Bank Soal	Mengakses berkas <code>questions.json</code> , mengacak soal, dan melakukan pencampuran pilihan jawaban.
data/questions.json	Database Soal JSON	Kumpulan 40 soal mitigasi lingkungan yang terbagi dalam 5 kategori edukasi iklim.

Diagram Navigasi Scene (State Transition)

Mesin Phaser mengelola pergantian status antar-scene secara efisien untuk menghindari kebocoran memori. Transisi scene diatur dengan skema berikut:

```

[ index.html ] -> Memulai main.js
    |
    v
[ MenuScene ] --- (Pilih Level: Mudah/Sedang/Sulit) ---> [ GameScene ]
                                     |
          (Awan Mendekat)          |          (Nyawa Habis)
          Launch (Overlay)         |          Transition Out
                |                   v
                v                   [ ScoreScene ]
[ QuestionScene ]                 |
    |                           | (Main Lagi / Retry)
    |                           v
    | (Selesai Jawab)           v
    | Resume & Stop --> [ GameScene ]

```

PENTINGNYA MODE OVERLAY

`QuestionScene` tidak mematikan `GameScene`. Ia diluncurkan menggunakan perintah `this.scene.launch()`, yang memungkinkannya berjalan sebagai lapisan overlay transparan di atas gameplay yang sedang berjalan tanpa menghapus cache keadaan permainan.

3. Analisis Mekanik Core Game

3.1 Lokasi Munculnya Musuh (Enemy Spawning)

Rintangan (musuh) muncul secara dinamis dari sisi kanan layar, tepatnya di luar batas pandang pemain (offset $X = \text{width} + 100$). Hal ini memberikan ilusi kepada pemain bahwa rintangan datang dari arah depan saat pesawat terbang maju. Titik koordinat vertikal (Y) diacak untuk memastikan rintangan tersebar rata di sepanjang jalur terbang.

Fungsi spawning diimplementasikan di dalam `src/scenes/GameScene.js` pada metode `_spawnObstacle()`:

```
// GameScene.js - Garis 320: Implementasi Spawning Rintangan
_spawnObstacle() {
  if (this.isGameOver || this.questionActive || this._paused) return;
  const { width, height } = this.scale;

  // Menentukan tipe rintangan (40% Khusus, 60% Biasa)
  const isSpecial = Math.random() < 0.4;
  const texture = isSpecial ? 'obstacle_cloud' : 'obstacle_factory';
  // Posisi Y diacak di antara batas terbang aman (80px s/d height - 80px)
  const spawnY = Phaser.Math.Between(80, height - 80);
  const scale = Phaser.Math.FloatBetween(0.12, 0.2);

  const obs = this.add.image(width + 100, spawnY, texture)
    .setScale(scale)
    .setDepth(4)
    .setTint(isSpecial ? 0xff8844 : 0xffffffff);

  // Menyimpan data rintangan ke array pelacak aktif
  const obsData = {
    img: obs,
    isSpecial,
    triggered: false,
    speed: this.gameSpeed * Phaser.Math.FloatBetween(0.8, 1.2),
    halfW: obs.displayWidth / 2,
    halfH: obs.displayHeight / 2
  };
  // ...
}
```

3.2 Mekanik Awan Besar (Special Obstacles)

Awan Besar (Awan Polusi) diidentifikasi dengan tekstur `obstacle_cloud`. Awan ini merupakan rintangan khusus yang tidak dapat dilewati secara normal begitu saja karena mengaktifkan modul soal. Secara visual, awan diberi warna jingga kecoklatan (tint `0xff8844`) yang melambangkan gas berbahaya, efek animasi berdenyut (pulsing tween), serta indikator teks tanda tanya (?) di atasnya untuk membedakannya dengan pabrik biasa.

```
// GameScene.js - Garis 345: Animasi khusus dan indikator Awan Besar
if (isSpecial) {
  // Efek berdenyut (pulsing alpha) agar awan terlihat dinamis dan berbahaya
  this.tweens.add({
    targets: obs,
    alpha: 0.7,
    yoyo: true,
    repeat: -1,
    duration: 500
  });

  // Tanda tanya melayang di atas awan
  const qMark = this.add.text(obs.x, obs.y - obs.displayHeight / 2 - 15, '?', {
    fontSize: '18px'
  }).setDepth(6);
  obsData.qMark = qMark;
}
```

3.3 Pengaturan Spawn Rate

Spawn rate rintangan dikontrol menggunakan sistem timer event dari Phaser (`Phaser.Time.Clock`). Durasi jeda pemanggilan fungsi `_spawnObstacle()` ditentukan berdasarkan parameter tingkat kesulitan yang dipilih oleh pemain pada awal permainan.

```
// GameScene.js - Garis 80: Konfigurasi jeda spawn berdasarkan kesulitan
const spawnDelayMap = { easy: 3000, medium: 2200, hard: 1500 };

this.obstacleTimer = this.time.addEvent({
  delay: spawnDelayMap[this.difficulty] || 2200, // jeda milidetik
  callback: this._spawnObstacle,
  callbackScope: this,
  repeat: -1 // Berulang selamanya
});
```

Pada tingkat kesulitan **Easy**, rintangan muncul setiap 3,0 detik, memberikan ruang gerak luas bagi pemain. Pada tingkat **Medium**, jeda dikurangi menjadi 2,2 detik. Sedangkan pada tingkat **Hard** rintangan muncul bertubi-tubi setiap 1,5 detik, memaksa pemain untuk bertindak cepat.

3.4 Sistem Pengaturan Kecepatan (Speed Scaling)

Sistem kecepatan game (`this.gameSpeed`) memiliki beberapa lapisan logika dinamis yang bertujuan meningkatkan tingkat ketegangan permainan dan kenyamanan menjawab soal:

1. **Kecepatan Awal (Base Speed):** Kecepatan awal pesawat dan latar belakang ditentukan oleh tingkat kesulitan game. Easy memiliki kecepatan dasar 220 px/dtk, Medium 300 px/dtk, dan Hard sebesar 400 px/dtk.
2. **Peningkatan Bertahap (Progressive Acceleration):** Kecepatan game ditingkatkan sebesar +20 px/dtk setiap 30 detik permainan berlangsung untuk menciptakan tantangan progresif. Nilai ini dibatasi maksimal 700 px/dtk agar game tetap dapat dimainkan secara manusiawi (Garis 748-751).

```
// Setiap 30.000 ms (30 detik), naikan kecepatan game
if (Math.floor(time / 30000) > this._lastSpeedIncrease) {
  this._lastSpeedIncrease = Math.floor(time / 30000);
  this.gameSpeed = Math.min(this.gameSpeed + 20, 700);
}
```

3. **Variasi Kecepatan Individual:** Setiap musuh yang muncul diacak kecepatannya berkisar antara 80% hingga 120% dari kecepatan game saat itu (`this.gameSpeed * FloatBetween(0.8, 1.2)`) agar tidak berbaris kaku.
4. **Sinematik Slow-Motion Saat Kuis:** Ketika ada soal aktif di layar (`this.questionActive == true`), kecepatan rintangan yang bergerak diperlambat menjadi 15% dari kecepatan aslinya. Hal ini mencegah rintangan menabrak pemain saat pemain sedang fokus membaca dan memilih jawaban soal (Garis 667-668).

```
// Memperlambat objek di Layar saat QuestionScene aktif  
const obsSpeed = this.questionActive ? this.gameSpeed * 0.15 : o.speed;  
o.img.x -= obsSpeed * dt;
```

3.5 Mekanik Tembakan & Amunisi Laser

Pesawat dilengkapi dengan meriam laser hijau ramah lingkungan (efek `0x86efac`) untuk menembak cerobong asap pabrik. Laser digambar secara manual di runtime menggunakan fungsi grafis (graphics rendering) Phaser, sehingga menghemat konsumsi memori pemuatan sprite eksternal.

Amunisi dibatasi sebanyak maksimal 5 tembakan (ditampilkan sebagai status `⚡ LASER: 5/5` di HUD). Pemain dapat menembak secara manual menggunakan tombol `SPACE` di keyboard komputer atau tombol merah virtual `TEBAK` pada perangkat layar sentuh/mobile.

```
// GameScene.js - Garis 435: Pembuatan Proyektil Laser secara dinamis
_shootBullet() {
  this.sound.play('laser', { volume: 0.65 });

  // Menggambar lingkaran inti peluru laser
  const bullet = this.add.graphics().setDepth(6);
  bullet.fillStyle(0x86efac, 1);
  bullet.fillCircle(0, 0, 7);
  bullet.x = this.player.x + this.player.displayWidth / 2;
  bullet.y = this.player.y;

  // Menggambar lingkaran cahaya aura (glow effect) di belakang peluru
  const glow = this.add.graphics().setDepth(5);
  glow.fillStyle(0x4ade80, 0.3);
  glow.fillCircle(0, 0, 14);
  glow.x = bullet.x;
  glow.y = bullet.y;

  this.bullets.push({ gfx: bullet, glow, vy: 0, active: true });
  // Efek memudar untuk glow proyektil
  this.tweens.add({ targets: glow, alpha: 0, duration: 600, onComplete: () => glow.destroy() });
}
```

Ketika amunisi habis (0/5), indikator teks laser di HUD akan berubah warna menjadi merah. Pengisian ulang (reload) amunisi hanya dapat dilakukan secara instan apabila pemain berhasil menjawab modul soal kuis dengan benar di layar.

3.6 Mekanik Kemunculan Soal (Question Triggering)

Awan besar (special obstacle) yang bergerak dari kanan tidak langsung memicu soal ketika pertama kali muncul di layar. Modul soal baru akan diaktifkan secara dinamis saat awan besar melewati sensor radar pesawat, yakni ketika koordinat X awan menyentuh batas 65% lebar layar (sekitar area `x < width * 0.65`).

```
// GameScene.js - Garis 715: Sensor pemicu modul kuis
if (o.isSpecial && !o.triggered && o.img.x < this.scale.width * 0.65) {
  this._triggerQuestion(o);
}
```

Metode `_triggerQuestion()` akan memanggil `QuestionManager` untuk mengambil satu soal acak dari berkas JSON, mengacak posisi opsi jawaban agar tidak mudah dihafalkan, lalu meluncurkan `QuestionScene` sebagai scene overlay dengan mengirimkan data-data tersebut.

3.7 Penalti Jawaban Salah & Tabrakan

Game ini memberikan konsekuensi taktis apabila pemain tidak berhasil menjawab soal dengan benar atau kehabisan waktu pengerjaan:

1. **Penalti Jawaban Salah / Waktu Habis:** Awan polusi besar tidak hancur, amunisi laser tidak diisi ulang, dan `QuestionScene` ditutup dengan lambat. Pemain kembali ke mode terbang manual dan dipaksa melakukan manuver menghindar (evasive action) agar awan polusi tersebut tidak menabrak pesawat (Garis 471).
2. **Tabrakan dengan Rintangan / Proyektil Gas Pabrik:** Pabrik biasa secara berkala meluncurkan awan gas abu-abu (gas projectile) ke arah kiri. Jika pesawat tertabrak oleh pabrik, awan besar, atau proyektil gas pabrik, pesawat akan meledak, layar berkedip merah (screen flash), pesawat berguncang (shake), dan nyawa berkurang 1 poin (Garis 399-433).

```
// GameScene.js - Garis 399: Logika pengurangan nyawa dan feedback visual
_loseLife() {
    if (this.isGameOver) return;
    this.lives--;

    // 1. Redupkan dan kecilkan ikon hati di HUD
    if (this.lives >= 0 && this.lives < 3) {
        const idx = this.lives;
        this.tweens.add({
            targets: this.lifeTexts[idx],
            alpha: 0.2,
            scaleX: 0.6,
            scaleY: 0.6,
            duration: 400
        });
    }

    // 2. Efek Layar berkedip merah (screen flash)
    const flash = this.add.graphics().setDepth(50);
    flash.fillStyle(0xff0000, 0.35);
    flash.fillRect(0, 0, this.scale.width, this.scale.height);
    this.tweens.add({ targets: flash, alpha: 0, duration: 400, onComplete: () => flash.destroy() });

    // 3. Efek guncangan pada gambar pesawat pemain (plane shake)
    this.tweens.add({
        targets: this.player,
        x: this.player.x + 15,
        yoyo: true,
        repeat: 3,
        duration: 60
    });

    // 4. Masuk ke Game Over jika nyawa habis
    if (this.lives <= 0) {
        this._gameOver();
    }
}
```

3.8 Sistem Nyawa & Damage Loop

Nyawa awal diatur sebanyak 3 poin (`this.lives = 3`). HUD di bagian atas kiri layar menampilkan 3 ikon hati () secara real-time. Jika nyawa mencapai nol, loop permainan utama dihentikan, semua event timer dihapus (menghindari spawn musuh baru), proyektil dibersihkan, efek audio baling-baling dimatikan, dan game beralih ke `ScoreScene` (Garis 494).

3.9 Konfigurasi Tingkat Kesulitan

Menu utama ([MenuScene](#)) menyediakan tiga protokol kesulitan yang dapat dipilih pemain. Pilihan ini akan memengaruhi tiga variabel penentu mekanik permainan secara simultan di [GameScene](#) :

1. **Timer Soal (Question Timer):** Menentukan batas waktu berpikir yang diberikan kepada pemain untuk menjawab soal di [QuestionScene](#) .
2. **Jeda Muncul Musuh (Spawn Delay):** Menentukan seberapa sering rintangan baru ditambahkan ke layar permainan.
3. **Kecepatan Awal (Base Speed):** Menentukan kecepatan scroll latar belakang serta pergerakan dasar rintangan.

Tabel Komparasi Pengaturan Parameter Tingkat Kesulitan

Tingkat Kesulitan	Kecepatan Awal	Jeda Spawning	Batas Waktu Soal	Akurasi Nyawa Target	Deskripsi Teknis UI
 EASY_MOD (Mudah)	220 px/dtk	3000 ms (3.0s)	15 Detik / Soal	Sangat Longgar	Pemain memiliki waktu berpikir panjang dan ruang navigasi yang sangat renggang.
 MED_MOD (Sedang)	300 px/dtk	2200 ms (2.2s)	10 Detik / Soal	Standar Balance	Kecepatan ideal dan direkomendasikan untuk melatih kemampuan motorik dan kognitif iklim.
 HARD_MOD (Sulit)	400 px/dtk	1500 ms (1.5s)	7 Detik / Soal	Sangat Ketat	Rintangan sangat rapat dan waktu menjawab sangat sempit, membutuhkan respons instan.

PENGARUH SKOR BONUS (TIME BONUS SCORE)

Selain parameter fisik game di atas, tingkat kesulitan secara tidak langsung menentukan tingkat perolehan skor bonus. Game memberikan tambahan skor **+50 poin** jika pemain berhasil menjawab dengan benar dalam waktu kurang dari 5 detik sejak soal terbuka. Pada mode Hard, sisa waktu bonus hanya berselisih 2 detik dari batas waktu maksimal (7 detik), membuat bonus ini sangat bergengsi.

4. Logika Game (Game Loop) & Detail Kecil

4.1 Scrolling Background & Parallax

Mekanik penerbangan tak terbatas (endless runner) disimulasikan menggunakan teknik tiling background horizontal sederhana. Terdapat dua gambar latar belakang langit yang sama (`this.bg1` dan `this.bg2`) yang ditempatkan bersebelahan.

Di dalam metode `update()`, kedua gambar digeser ke arah kiri secara terus-menerus. Ketika gambar pertama telah bergeser sepenuhnya keluar dari sisi kiri layar, posisinya segera dipindahkan ke sebelah kanan gambar kedua, menciptakan perputaran tak terbatas (endless seamless looping) tanpa putus.

```
// GameScene.js - Garis 592: Logika Seamless Loop Background
const bgMoveSpeed = this.gameSpeed * 0.35; // Latar bergerak Lebih Lambat (efek kedalaman/parallax)
const bgScaledW = this.bg1.width * this.bg1.scaleX;

this.bg1.x -= bgMoveSpeed * dt;
this.bg2.x -= bgMoveSpeed * dt;

// Jika bg1 sudah keluar layar ke kiri, pindahkan ke kanan bg2
if (this.bg1.x + bgScaledW <= 0) {
    this.bg1.x = this.bg2.x + bgScaledW;
}
// Jika bg2 sudah keluar layar ke kiri, pindahkan ke kanan bg1
if (this.bg2.x + bgScaledW <= 0) {
    this.bg2.x = this.bg1.x + bgScaledW;
}
```

4.2 Hovering Physics & Kontrol Pesawat

Pesawat tidak menggunakan sistem gravitasi berat bawaan Phaser Arcade Physics agar gerakan terasa ringan seperti pesawat layang futuristik. Kecepatan vertikal (`this.playerVY`) diatur secara manual berdasarkan input pemain (W/S atau Panah Up/Down).

Ketika tombol dilepas, pesawat tidak langsung berhenti mendadak, melainkan melambat secara halus menggunakan faktor redaman (damping factor) sebesar `0.85`, memberikan kesan melayang (hovering) yang realistis. Selain itu, pesawat dimiringkan ke atas (-12 derajat) saat naik dan dimiringkan ke bawah (+12 derajat) saat turun untuk memberikan umpan balik visual gerak yang halus.

```
// GameScene.js - Garis 606: Kontrol penerbangan dan efek hovering
if (goingUp) {
    this.playerVY = -moveSpeed;
    this.player.setAngle(-12); // Miring ke atas
} else if (goingDown) {
    this.playerVY = moveSpeed;
    this.player.setAngle(12); // Miring ke bawah
} else {
    this.playerVY *= 0.85; // Damping - efek melambang melambat halus
    this.player.setAngle(0); // Lurus kembali
}
```

4.3 Efek Ledakan Partikel Vektor

Ketika rintangan hancur tertembak laser atau menabrak pemain, game tidak memuat aset gambar ledakan sprite sheet yang berat. Sebaliknya, game menggambar partikel vektor secara dinamis menggunakan class grafis Phaser.

Metode `_explodeObstacle()` menghasilkan 8 lingkaran partikel kecil berwarna cerah (oranye, merah, kuning, dan hijau) secara acak yang dilontarkan ke segala arah sudut menggunakan fungsi trigonometri sinus dan kosinus, kemudian memudar perlahan (fade out) menggunakan tweening.

```
// GameScene.js - Garis 754: Pembuatan Ledakan partikel vektor
_explodeObstacle(x, y) {
  for (let i = 0; i < 8; i++) {
    const particle = this.add.graphics().setDepth(15);
    // Memilih warna partikel secara acak
    particle.fillStyle(Phaser.Math.RND.pick([0xff8c00, 0xff4444, 0xffff00, 0x4ade80]), 1);
    particle.fillCircle(0, 0, Phaser.Math.Between(4, 10));
    particle.x = x;
    particle.y = y;

    const angle = (i / 8) * Math.PI * 2; // Membagi lingkaran menjadi 8 arah
    const speed = Phaser.Math.Between(60, 150);

    // Animasi melempar partikel keluar sambil mengecil dan memudar
    this.tweens.add({
      targets: particle,
      x: x + Math.cos(angle) * speed,
      y: y + Math.sin(angle) * speed,
      alpha: 0,
      scaleX: 0.1,
      scaleY: 0.1,
      duration: Phaser.Math.Between(400, 700),
      onComplete: () => particle.destroy()
    });
  }
}
```

4.4 Sistem Pause Jeda Game

Fasilitas menjeda game diimplementasikan dengan menekan tombol `P` atau ikon jeda di HUD. Jeda game akan membekukan seluruh pergerakan musuh, menghentikan akumulasi skor otomatis, menjeda efek audio putaran rotor pesawat (`balancing`), dan menampilkan layar hitam transparan redup (overlay) dengan teks panduan instruksi (Garis 535-578).

4.5 Detail Kecil Lainnya

- **Floating Score Indicator (Skor Melayang):** Ketika pemain berhasil menghancurkan musuh atau menjawab benar, indikator teks melayang (contoh: `+100` berwarna kuning emas) akan muncul di

dekat pesawat dan bergerak naik perlahan sebelum memudar dan terhapus dari memori (Garis 372).

- **Bobbing Effect (Efek Mengapung):** Saat pesawat diam tidak digerakkan ke atas/bawah, pesawat akan melayang dengan gerakan mengapung naik-turun yang halus secara otomatis menggunakan persamaan matematika gelombang sinus: `this.player.y += Math.sin(time / 600) * 0.4` (Garis 622).
- **BGM Looping & State Check:** Game memeriksa status pemutaran musik latar untuk mencegah musik berlipat ganda saat pemain menekan tombol ulang (retry) permainan.

5. Kesimpulan & Ringkasan Teknis

Game **Bhumi Raksha: Skies of Tomorrow** berhasil menyajikan sebuah media multimedia interaktif yang edukatif, modern, dan sangat responsif. Melalui integrasi Phaser.js 3 dan logika modular JavaScript, game dapat dioptimalkan dengan baik untuk dimainkan di perangkat desktop maupun mobile secara lancar.

Ringkasan Nilai Parameter Teknis Utama

Komponen Logika	Nama Metode / Variabel	Parameter Nilai Default	Fungsi Utama dalam Game
Dimensi Layar	<code>config.width / height</code>	960px x 640px	Resolusi kanvas game aspek rasio 3:2.
Nyawa Pemain	<code>this.lives</code>	3 Poin Nyawa	Batas ketahanan tabrakan sebelum Game Over.
Kapasitas Laser	<code>this.bulletsLeft</code>	5 Amunisi (Maksimal)	Membatasi tembakan manual pemain di layar.
Skor Kelangsungan Hidup	<code>this.scoreTick</code>	+5 Poin / Detik	Skor pasif yang diperoleh selama bertahan hidup.
Skor Jawaban Benar	<code>_onQuestionResolved()</code>	+100 Poin (Dasar)	Skor utama jika berhasil menjawab modul soal kuis.
Bonus Respons Cepat	<code>gotBonus</code>	+50 Poin (Tambahan)	Diberikan jika menjawab soal di bawah 5 detik.
Perlambatan Kuis	<code>this.questionActive</code>	15% dari Kecepatan Normal	Mencegah tabrakan saat popup soal aktif.
Faktor Damping	<code>this.playerVY damping</code>	Faktor pengali 0.85	Menghasilkan perlambatan gerak hover yang mulus.
Sudut Kemiringan	<code>setAngle()</code>	-12° (Naik) / +12° (Turun)	Efek visual hidrodinamis saat manuver pesawat.

- AKHIR DARI BUKU PANDUAN TEKNIS -

Bhumi Raksha: Skies of Tomorrow · Project Green Ascent · 2026